PilotFish Http Response Headers Processor – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

When a route answers an HTTP client synchronously, this step **adds response headers** — things like `Content-Type` or custom tracing IDs — by merging a configured name/value table into a shared map on the transaction. The HTTP listener reads that map when it sends the reply.

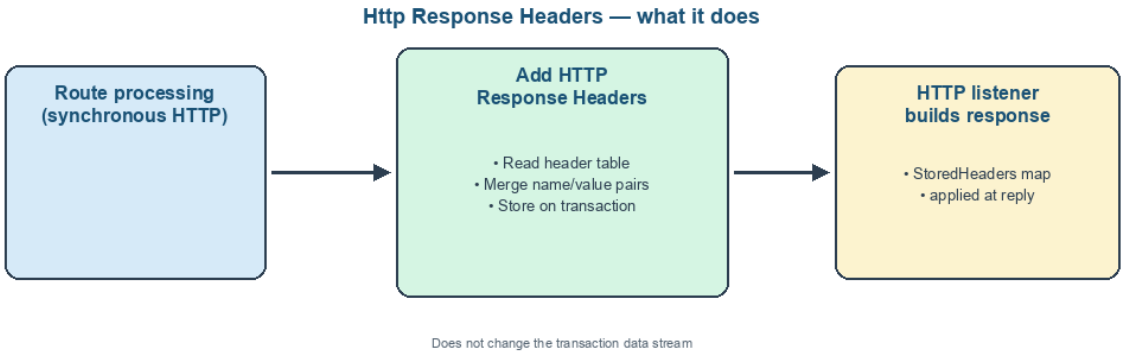

Figure 1: At a glance (plain English)

- **Merges** configured header rows into `StoredHeaders` on the transaction.
- **Preserves** headers already set by earlier stages.
- **Does not** change the response body stream.
- **Pairs with** synchronous HTTP listeners and status-code processors.

Purpose

This document is a **deep dive** into the **Http Response Headers** processor as shipped in `eip.war.hs.26R1.11`: how it accumulates HTTP response headers on a transaction attribute for synchronous HTTP reply paths.

Provenance	Value
WAR	`eip.war.hs.26R1.11`
Module JAR	`xcs-modules-http-1.0.3.jar`
Decompiler	CFR 0.152
Implementation class	`com.pilotfish.eip.modules.http.AddHttpResp`
Decompiled path	`war-pipeline/decompiled/eip-26R1.11/.../Ad`
Processor type string	`Http Response Headers`
Description	`Adds headers to the stored headers attribute for HTTP response headers`
Help ID	`console.processor.http_response_header`
Categories	`HTTP`, `WEB`

---

1. Conceptual model

```
Earlier stages may set StoredHeaders (HashMap)
        |
        v
  Add Http Response Headers
        |
        +-- read HeaderTable rows (Name / Value)
        +-- storedHeaders.put(name, value) for each row
        |
        v
  HTTP listener (synchronous response)
        reads StoredHeaders + body stream + HTTPResponseCode
```

This processor is **metadata-only** — it never reads or writes the transaction data stream.

---

2. High-level behavior (`processData`)

1. Read `TransactionAttributes` from `data`.
2. Look up `com.pilotfish.eip.modules.http.StoredHeaders` (`STORED_HEADERS_ATTR`).
   - If missing, create a new `HashMap<String, String>` and store it on the transaction.
3. Parse the **Headers** table via `manager.getParsedStringValue(data, "HeaderTable")` and `TableConfigurationItem.convertInputToMetaObject(...)`.
4. For each `Pair` in the list:

- `storedHeaders.put(headerName, headerValue)` — **last write wins** for duplicate names in this table pass.

5. Return the same `TransactionData`.

If the table converts to `null`, no rows are added (existing map unchanged).

---

3. Configuration reference

3.1 Headers

Property	Value
UI label	Headers
Tag	`HeaderTable`
Type	Table (Name / Value columns)
Default	empty
Tooltip	(table item — header name/value pairs)

Nuances:

- Values pass through `getParsedStringValue`, so expression substitution and enhanced expressions apply like other table-driven modules.
- Multiple processor instances in one route **accumulate** into the same map attribute.
- Header names are not normalized (case sensitivity depends on the HTTP listener implementation).

3.2 Inherited AbstractProcessor options

Property	Value
Tag	`ExecuteProcessor`
Default	`true`
Tab	Conditional Execution

When skipped, the `StoredHeaders` map is not created or updated by this step.

---

4. Transaction attributes

4.1 Written / updated

Attribute key	Type	Meaning
`com.pilotfish.eip.modules.http.StoredHeaders`	`HashMap<String, String>`	Cumulative HTTP response headers

4.2 Related attributes (other modules)

Attribute	Set by
`com.pilotfish.HTTPResponseCode`	**Http Response Status Code** processor

HTTP listeners consume these when completing a synchronous request.

---

5. Worked example

Route: RESTful Web Service Listener (synchronous) -> transform -> **Add Http Response Headers** -> **Http Response Status Code**

Header name	Header value
`Content-Type`	`application/xml`
`X-Request-Id`	`%com.acme.RequestId%`

After execution, `StoredHeaders` contains both entries. The listener applies them with the configured status code when returning the body stream.

---

6. Known quirks and caveats

1. **No stream interaction** — body content must come from other processors.
2. **Map merge semantics** — `put` overwrites same key; order is table iteration order within this processor, plus any prior map contents.
3. **Not registered via `registerTransactionAttribute`** — attribute is conventional but widely used across HTTP modules.
4. **Empty table is a no-op** except ensuring map creation when attribute was absent and processor runs.

---

7. Operational recommendations

Goal	Guidance
Synchronous APIs	Place after the body is finalized, before transport/listener reply
Content-Type	Set explicitly when the body is XML/JSON/text
Security headers	Add `Cache-Control`, `X-Frame-Options`, etc. here

Goal	Guidance
Reuse	Split common headers into a shared sub-route processor chain
Status codes	Combine with **Http Response Status Code** processor

---

8. Source index (this WAR)

Topic	Path under `war-pipeline/decompiled/eip-26R1.11/`
Add HTTP Response Headers	`com/pilotfish/eip/modules/http/AddHttpResponseHea`
AbstractProcessor	`com/pilotfish/eip/extend/AbstractProcessor.java`
Module JAR	`war-pipeline/jars/eip-26R1.11/xcs-modules-http-1.`

---

9. Pipeline provenance

```
PilotFish WARs/eip.war.hs.26R1.11
  -> xcs-modules-http-1.0.3.jar
  -> CFR decompile
  -> Documents/Processors/26R1.11/PilotFish-Http-Response-Headers-Reference-26R1.11.(md|pdf)
```

Deep-dive documentation from WAR decompile – Http Response Headers Processor – 26R1.11 – August 2026.